Weekly update 8

DRIFT: Detecting and Retiring Invalidated Facts over Time

Pipeline, extraction and retrieval against Zep, multi-agent related work, correlations for RQ4, and the plan for three runs

Nandan Hemanth · 5 November 2026

DRIFT: Detecting and Retiring Invalidated Facts over Time

QUESTION when a fact an agent remembered is replaced, how should memory forget the old one, and how much does that choice matter?

Detecting
noticing that a new fact replaces a stored one: same subject and relation, new value

Retiring
hiding the replaced fact from retrieval at once, deleting it after a grace period

Invalidated facts
facts whose validity has closed: still stored, no longer current

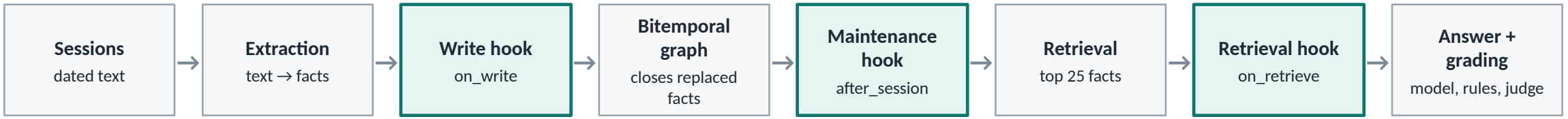
over Time
two timelines per fact: when it was true, and when the agent learned it

held fixed in every experiment	varied	measured
memory store, extraction, retrieval, answer prompt, judge	the forgetting policy (7 policies)	accuracy, stale answers, tokens, facts stored

Example: 'I report to Ashley' (March), then 'Ashley left, I report to Daniel' (July). Both are stored and both match 'who do I send my report to?'. DRIFT detects that the second replaces the first and retires the first, so the model only sees Daniel.

The pipeline, and where each policy acts

ONE POLICY PER RUN every experiment runs the whole pipeline with exactly one policy switched in. The three hooks always exist; only the chosen policy's hooks do anything, the rest pass facts through unchanged.



policy (one per run)	write hook	maintenance hook	retrieval hook	what it does
NOMEM			returns nothing	no memory: the floor
P0 append-only				keeps and shows every fact
P1 time decay		deletes facts > 20 days		forgets by age
P2 bitemporal		**deletes 7 days after closing**	**hides closed facts**	forgets what was replaced
P3 novelty gate	refuses near-duplicates			forgets before storing
P4 LLM consolidation		LLM drops facts (every 3rd session)		forgets what the LLM disbelieves
P5 topology		keeps best-supported value		forgets by graph support, no dates

Closing a replaced fact happens in the store for every policy, P0 included; only the hooks hide or delete.

Extraction: how DRIFT and Zep turn conversation into facts

EXTRACTION the LLM step that converts conversation text into stored facts; it is identical for every policy, so it cannot explain differences between policies.

	Zep (Rasmussen et al., 2025, Sec. 2.2)	Mem0 (Chhikara et al., 2025, Sec. 2)	DRIFT
unit stored	entities + fact edges between them	short natural-language facts	triples: subject | relation | object
context given	current message + previous 4 messages	conversation summary + last 10 messages	the session's lines only
model	gpt-4o-mini	gpt-4o-mini	DeepSeek V4.1 Flash, one for all cells
checks	reflection step; LLM entity resolution	LLM: ADD / UPDATE / DELETE / NOOP	rule pass for single-valued facts; retry if unreadable
dates	LLM extracts valid_at / invalid_at	not described (base); relations marked invalid (graph variant)	valid-from = session date

Should DRIFT add the previous 4 messages, like Zep? On MQuAKE-T and FactConsolidation, each line is a self-contained fact, so earlier messages add nothing. On ConvoMem, values are revised in passing ('let's do Saturday instead'), so context should help: test it as a separate ConvoMem extraction study, not inside the main comparison.

How each retrieval signal is calculated, and how Zep retrieves

EXAMPLE QUESTION 'What is the name of the current head of the New Zealand government?' Fact: 'The head of government of New Zealand is Chris Hipkins.'

Semantic (meaning)

Formula: cos(q, f) = (q · f) / (|q| |f|), where q and f are the embeddings of question and fact (nomic-embed-text).

Range: −1 to 1 in theory; about 0 to 1 for text. Unitless.

Source: cosine relevance in Generative Agents (Park et al., 2023, Sec. 4.1) and A-MEM (Xu et al., 2025, Eq. 9); Zep and Mem0 search the same way.

Lexical (shared words)

Formula: |Q ∩ F| / |Q|, the share of the question's words (3+ letters) found in the fact.

Example: 8 question words, 5 in the fact (the, head, government, new, zealand): 5/8 = 0.63.

Range: 0 to 1. Unitless.

Source: a simplified form of BM25 (Robertson and Zaragoza, 2009), which Zep uses; no word-rarity weighting.

Recency (time)

Formula: (t_f − t_min) / (t_max − t_min), where t_f is the date the fact became true.

Example: oldest fact in memory scores 0, newest 1, a fact halfway through scores 0.5.

Range: 0 to 1. Unitless.

Source: min-max-normalised recency in Generative Agents (Park et al., 2023, Sec. 4.1), which decays by 0.995 per hour since last access.

	Zep (Rasmussen et al., 2025, Sec. 3)	DRIFT
search	cosine (BGE-m3), BM25 full-text, breadth-first graph search	cosine, word overlap, recency
combine	rerank: Reciprocal Rank Fusion, score = Σ 1 / (k + rank) (Cormack et al., 2009); also MMR, cross-encoder	0.6 x semantic + 0.3 x lexical + 0.1 x recency
prompt	top 10 facts with their valid date ranges	top 25 facts with their valid-from dates

Retrieval: what Zep does and what DRIFT does

HYBRID RETRIEVAL combining a meaning-based signal with a word-based one; standard in agent memory. The difference is how the signals are combined.

Zep (Rasmussen et al., 2025, Sec. 3)

- **Three searches:** cosine similarity (BGE-m3 embeddings), BM25 full-text, breadth-first graph search.
- **Combined by reranking, not weights:** Reciprocal Rank Fusion, MMR, graph distance or a cross-encoder.
- **Time:** graph search seeded with recent episodes; fact date ranges shown in the prompt.
- **Top 10** results go to the model.

DRIFT

- **Three signals:** cosine similarity (nomic-embed-text), word overlap, recency.
- **Combined by a weighted sum:** 0.6 semantic + 0.3 lexical + 0.1 recency.
- **Time:** explicit recency signal; each fact's date shown in the prompt.
- **Top 25** results go to the model. RRF is implemented as a weight-free alternative.

Why DRIFT uses weights: a single weighted formula makes each signal's influence measurable, which is what allowed the 1,000-weighting sweep; rank fusion has nothing to vary. Next comparison: DRIFT with RRF (Cormack et al., 2009), end to end, against 0.6 / 0.3 / 0.1.

The three retrieval signals: range, units, and whether the weights must sum to 1

SCORE score = w_sem x semantic + w_lex x lexical + w_rec x recency; every signal is a unitless number between 0 and 1.

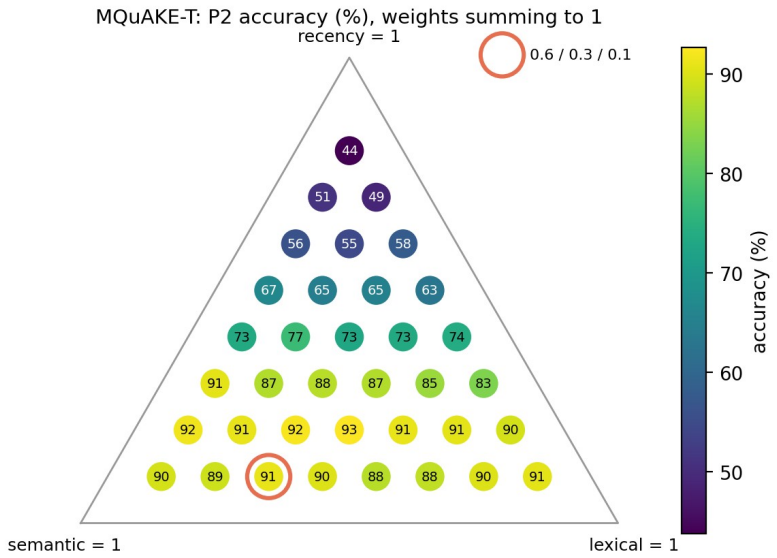
signal	exact definition in DRIFT	range	standard it follows
semantic	cosine of the angle between the question's and the fact's embeddings	0 to 1 for text (−1 to 1 in theory)	cosine relevance: Generative Agents (Park et al., 2023); A-MEM (Xu et al., 2025)
lexical	share of the question's words (3+ letters) that appear in the fact	0 to 1	simplified word matching; the standard full form is BM25 (Robertson and Zaragoza, 2009), used by Zep
recency	fact's valid-from date rescaled: oldest fact 0, newest 1 (min-max)	0 to 1	min-max-normalised recency: Generative Agents (Park et al., 2023), which uses exponential decay

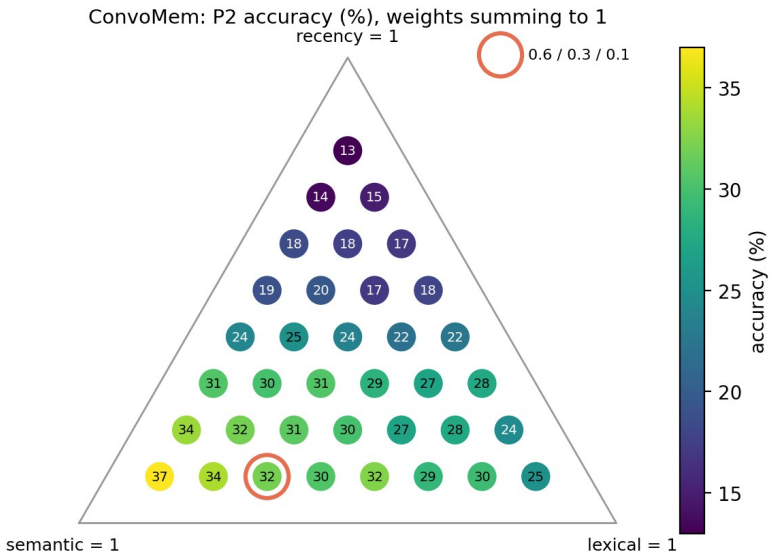
Must the weights sum to 1? No. Only the ranking of facts matters, and multiplying all weights by the same number leaves the ranking unchanged: (0.6, 0.3, 0.1) and (1.2, 0.6, 0.2) give the same top 25. Generative Agents sets all three weights to 1 (sum 3).

Checked in the data: the 1,000-weighting sweep contains 115 groups of proportional weightings; every group gave identical prompts and identical accuracy. Weights are reported rescaled to sum to 1 for readability.

Weightings that sum to 1: same results, 0.6 / 0.3 / 0.1 near the top

36 WEIGHTINGS every grid weighting whose three weights sum to 1 (each 0.1 to 0.8), P2, mean of Llama 3.1 8B and Qwen3 8B.





	MQuAKE-T	ConvoMem
0.6/0.3/0.1	**91.1%**	**32.0%**
rank of 36	4 to 6 (tie)	5 to 6 (tie)
best	92.7%	37.0%
worst	43.8%	13.0%
mean	78.3%	25.4%

Reading the triangle: each corner is one signal at weight 1. Accuracy is high along the bottom (little recency) and falls toward the recency corner. Not yet tested: a weight of 0 (for example no recency at all).

All 36 weightings that sum to 1, ranked by MQuAKE-T accuracy

P2 ACCURACY mean of Llama 3.1 8B and Qwen3 8B. Weights: semantic / lexical / recency. The thesis setting is highlighted.

#	sem.	lex.	rec.	MQuAKE-T	ConvoMem
1	0.4	0.4	0.2	92.7%	30.0%
2	0.5	0.3	0.2	92.2%	31.0%
3	0.7	0.1	0.2	91.7%	33.5%
4	0.6	0.2	0.2	91.1%	32.0%
5	**0.6**	**0.3**	**0.1**	**91.1%**	**32.0%**
6	0.3	0.5	0.2	91.1%	27.0%
7	0.6	0.1	0.3	90.6%	30.5%
8	0.2	0.6	0.2	90.6%	27.5%
9	0.1	0.8	0.1	90.6%	25.0%
10	0.5	0.4	0.1	90.1%	30.5%
11	0.2	0.7	0.1	90.1%	29.5%
12	0.8	0.1	0.1	89.6%	37.0%
13	0.1	0.7	0.2	89.6%	24.5%
14	0.7	0.2	0.1	89.1%	34.0%
15	0.3	0.6	0.1	88.0%	29.0%
16	0.4	0.5	0.1	87.5%	32.5%
17	0.4	0.3	0.3	87.5%	30.5%
18	0.5	0.2	0.3	87.0%	30.0%
19	0.3	0.4	0.3	87.0%	28.5%
20	0.2	0.5	0.3	85.4%	27.0%
21	0.1	0.6	0.3	83.3%	27.5%
22	0.4	0.2	0.4	77.1%	25.0%
23	0.1	0.5	0.4	74.0%	22.5%
24	0.5	0.1	0.4	73.4%	24.0%
25	0.2	0.4	0.4	73.4%	22.0%
26	0.3	0.3	0.4	72.9%	23.5%
27	0.4	0.1	0.5	66.7%	19.0%
28	0.2	0.3	0.5	65.1%	17.0%
29	0.3	0.2	0.5	64.6%	19.5%
30	0.1	0.4	0.5	63.0%	18.0%
31	0.1	0.3	0.6	57.8%	17.0%
32	0.3	0.1	0.6	56.2%	18.5%
33	0.2	0.2	0.6	54.7%	17.5%
34	0.2	0.1	0.7	51.0%	13.5%
35	0.1	0.2	0.7	49.0%	15.0%
36	0.1	0.1	0.8	43.8%	13.0%

Related work: memory shared by several agents (RQ4)

GAP current work on shared agent memory studies access control, provenance and design; none compares forgetting policies or tests whether their ranking holds across agents.

work	what it studies	relation to DRIFT
GateMem (Ren et al., 2026)	benchmark: utility, access control, forgetting after deletion, several principals	shared memory, but governance, not staleness
Governed Shared Memory (Margalit et al., 2026)	names stale propagation and contradiction persistence; temporal supersession in production	found a near-duplicate gate rejecting contradictory writes, as P3 does
Collaborative Memory (Rezazadeh et al., 2025)	private and shared memory tiers with provenance and access control	DRIFT also records provenance per fact
G-Memory (Zhang et al., NeurIPS 2025)	hierarchical memory graphs for multi-agent task success	no forgetting or staleness measured
Yu et al. (2026), position paper	multi-agent memory as a computer-architecture problem	calls memory consistency the most pressing open challenge

What DRIFT adds: the same seven policies run in three topologies (shared, independent, orchestrator); the policy ranking is compared with the single-agent one, and contradictions between different writers are counted.

RQ4: three correlation measures, what each tests, and the values

COMPARED single-agent policy ranking (MQuAKE-T accuracy, mean of 5 models) against each topology, 6 policies (P0 to P5). p = exact permutation test over all 720 orderings of the 6 policies.

	Pearson r (Pearson, 1896)	Spearman ρ (Spearman, 1904)	Kendall τ (Kendall, 1938)
compares	the accuracy values	the ranks	pairs of policies
formula	$\mathrm{cov}(x, y) / (\sigma x\, \sigma y)$	$1 - 6\Sigma d^2 / (n(n^2 - 1))$	(concordant − discordant) / $n(n-1)/2$
question it answers	do the scores rise and fall together in a straight line?	is the order the same? big rank jumps count more	in how many of the 15 pairs is the order kept?
independent	0.938 (p = 0.008)	0.943 (p = 0.017)	0.867 (p = 0.017)
orchestrator	0.972 (p = 0.006)	0.928 (p = 0.022)	0.828 (p = 0.028)
shared	0.893 (p = 0.022)	0.928 (p = 0.022)	0.828 (p = 0.028)

Why Spearman leads, Kendall checks, Pearson is context: RQ4 asks whether the order holds, so a rank measure fits; Pearson is affected by the lower overall accuracy with several agents. Kendall is more reliable for small samples with ties (6 policies). All three agree. None of the reference papers uses rank correlation; the method comes from the statistics papers cited.

One run or three? What the reference papers did

TEMPERATURE 0 asks the model for its most likely answer every time; hosted models are close to repeatable at 0, but not guaranteed identical.

paper	runs	setting
MemGPT (Packer et al., 2023)	1	not reported
Zep (Rasmussen et al., 2025)	1	no variance reported
LoCoMo (Maharana et al., 2024)	1	temperature 0, top-p 1
LongMemEval (Wu et al., 2025)	1	greedy decoding
Supersede (Patel, 2026)	1	temperature 0; re-evaluation reproduces the same count
Mem0 (Chhikara et al., 2025)	**10 (judge)**	temperature 0; SD 0.1 to 0.4 points

- **DRIFT so far:** 1 run, temperature 0, every call cached, so re-running reproduces every number exactly.
- **What could vary:** a fresh, uncached call to a hosted model, especially reasoning models.
- **Why the main results should hold:** P2 vs P0 won 26 questions and lost 2; P2 vs P5 won 113 and lost 0.

Plan: 3 runs in total. Runs 2 and 3 repeat the final run with the cache bypassed and nothing else changed. Report each result as mean ± SD over 3 runs, plus the share of questions whose verdict flips between runs.

Plan for the three runs, and next steps

RULE a repeat run changes nothing but the randomness; any design change (such as more extraction context) is a separate study.

study	what changes	what it shows	scope
runs 2 and 3 (repeats)	nothing; cache bypassed	how stable the results are (mean ± SD, flip rate)	all 105 cells, extraction to grading
Zep-style extraction context	extractor sees the previous 4 messages	whether ConvoMem's extraction ceiling rises	ConvoMem only
RRF retrieval	rank fusion instead of weights	how 0.6 / 0.3 / 0.1 compares with the standard method	P2, two small models
zero weights	for example recency = 0	whether recency is needed at all	P2, two small models

Thesis: writing Chapters 1 to 3 in Overleaf; Chapter 1 follows the Stevens template sections with citations placed per claim.

Questions for the committee: are three runs of the full matrix the right scope for the stability check, and should the extraction-context study be included or left as future work?